

First Assignment - Computer Aided Simulation Lab

Flavia Calfapietro
s345862@studenti.polito.it
Politecnico di Torino, Italy

November 6, 2025

1 Introduction

This project presents a Python-based simulator designed to analyze the operation of an urban electric car-sharing system. The main objective is to investigate how key parameters—such as fleet size, station capacity, user arrival rate, and relocation strategies—affect overall system performance, including service rate, vehicle availability, waiting times, and utilization. The simulator reproduces realistic urban mobility dynamics by modeling stochastic user demand, vehicle allocation, charging processes, and relocation within a 10×10 km area with spatially varying demand and time-dependent traffic conditions. An event-driven simulation architecture ensures efficient processing of user arrivals, trip completions, and charging events. Finally, a method for automatically detecting when the system reaches steady-state was implemented and validated on the car-sharing scenario, enabling a robust statistical analysis of performance metrics under near-stationary traffic conditions.

2 Lab 2 and 3— Modelling and Developing a Car-Sharing System

2.1 Key Parameters and Assumptions

The car-sharing simulation models an urban electric vehicle system within a 10×10 km city grid. Vehicle demand is not uniform: it is concentrated in five main hotspots, representing a central business district and four peripheral residential or commercial zones. This spatial clustering captures realistic patterns of urban mobility and reflects areas where users are most likely to request vehicles.

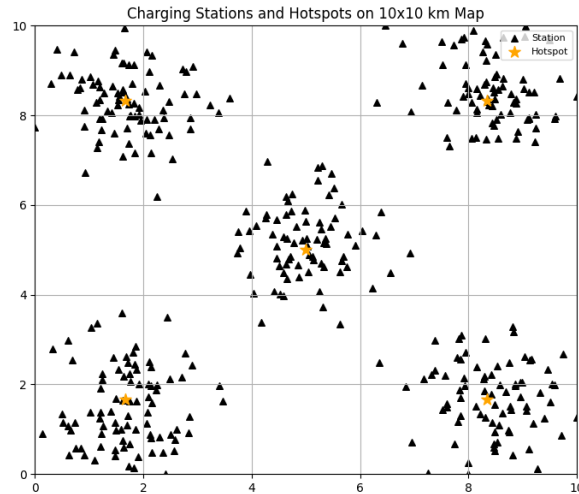


Figure 1: Fleet state distribution.

Each hotspot (x_h, y_h) was randomly placed on the map according to a uniform distribution over the map boundaries:

$$(x_h, y_h) \sim \text{Uniform}(0, \text{MAP_SIZE})$$

where Map.Size represents the length of the square simulation area in kilometers. Around each hotspot, user starting locations were sampled from a bivariate normal distribution with standard deviation $\sigma_h = 0.5$ km to model the spatial spread of demand:

$$(x, y) \sim \mathcal{N}((x_h, y_h), \sigma_h^2)$$

With this approach, users are more likely to appear near the hotspots, but some fraction of trips are generated randomly across the map to model background demand.

Traffic conditions vary throughout the day, with defined peak hours in the morning and evening where vehicle movement is slower due to congestion. Vehicles travel at a modest average speed, reflecting typical intra-city conditions, and their actual travel distances deviate from straight-line paths because of road layouts and detours. Users are assumed to have a limited walking tolerance to reach vehicles and accept proposed trips probabilistically. The simulation is event-driven: the system state updates only at discrete events such as user arrivals, trip completions, charging sessions, and scheduled fleet relocations. This approach ensures realistic temporal dynamics while maintaining computational efficiency.

2.1.1 Core Classes in the Simulator

The simulation is implemented using object-oriented design. The main classes represent vehicles, charging stations, and users.

Vehicle Class. Each vehicle object tracks its current location, battery level, operational state (**free**, **in-use**, **charging**, or **unavailable**), and destination. Vehicles are updated dynamically during events such as trip start, trip end, and charging completion.

Charging Station Class. Stations store their position, maximum capacity of four vehicles and a list of currently charging vehicles. They manage availability and queueing logic for vehicles that need to recharge.

User Class. User objects encapsulate arrival times, requested trips, walking tolerance, and the probabilistic acceptance of vehicle proposals. Each user interacts with the fleet through the vehicle assignment logic implemented in the simulation.

2.1.2 Performance metrics

The main performance metrics include:

- **Service Rate** $R_s = \frac{N_{\text{served}}}{N_{\text{total}}}$, the fraction of user requests fulfilled.
- **Availability Rate** $R_a = \frac{N_{\text{free}}}{N_{\text{total}}}$, representing the proportion of time vehicles are ready for assignment.
- **Utilization Rate** $R_u = \frac{N_{\text{in-use}}}{N_{\text{total}}}$, the fraction of the fleet actively serving trips.
- **Average Waiting Time** $E[T_{\text{wait}}] = E[T_{\text{walk}} + T_{\text{assign}}]$, including both walking and system delays.
- **Trip distance and duration:** The expected trip duration is computed as

$$E[T_{\text{trip}}] = \frac{E[d_{\text{real}}]}{v_{\text{avg}}},$$

where:

- d_{real} is the *actual travel distance*, which accounts for the straight-line distance between origin and destination multiplied by a stochastic *detour factor* to model route inefficiencies (e.g., street layout, turns, etc.).
- v_{avg} is the *average vehicle speed*, adjusted with a stochastic *traffic variability factor* to reflect random traffic delays.

This formulation provides the expected travel time for a trip, considering that vehicles may deviate from straight paths and that traffic conditions vary randomly.

2.2 Model Design and Code Functioning

The simulator models key dynamics of an urban electric car-sharing system, balancing realism and computational simplicity.

Arrival Function. The `arrival` process models the generation of new user requests in the car-sharing system. User arrivals are modeled as a time-dependent Poisson process, where inter-arrival times follow an exponential distribution with a rate parameter that varies over the day to reflect changing traffic demand:

$$\lambda_t = \begin{cases} \lambda \cdot 1.5 & \text{for peak hours (8–10, 18–20),} \\ \lambda \cdot 1.0 & \text{for midday hours (10–18),} \\ \lambda \cdot 0.6 & \text{for off-peak hours (nighttime and early morning).} \end{cases} \quad (1)$$

Upon arrival, each user seeks a vehicle within a maximum walking distance (`PICKUP_RADIUS_KM`) and with sufficient battery to complete the trip. Vehicle assignment is determined by a weighted scoring function, defined as:

$$S_v = w_b \cdot B_v + w_d \left(1 - \frac{d_v}{r_{\text{max}}} \right),$$

where:

- B_v is the **current battery level** (%) of vehicle v ;
- d_v is the **distance** (in km) between the vehicle and the requesting user;
- r_{max} is the **maximum pickup radius** within which a vehicle can be considered;
- w_b and w_d are the **weight coefficients**. In this case w_b is 0.4 and w_d is 0.6.

This formulation ensures that vehicles with a higher remaining charge and shorter distance to the user are prioritized. After computing the score for all available vehicles, the system selects the one with the highest S_v as the candidate. **Weight coefficients** are used to balance the importance of battery level and proximity in vehicle selection. In this configuration, $w_b = 0.4$ and $w_d = 0.6$, meaning that closer vehicles are given higher priority compared to those with higher battery levels.

Probabilistic approach Furthermore, user acceptance is modeled probabilistically through a parameter **acceptance probability** = 0.8, representing the likelihood that a user accepts the assigned vehicle. If the user refuses, a new vehicle may be proposed with a **retry probability**=0.6, allowing the system to attempt reassignment before the request is definitively dropped.

If the total walking time plus system delay exceeds a predefined threshold (`MAX_WAIT_MIN`), the request is canceled, representing user abandonment due to excessive waiting. This probabilistic decision layer adds realism to the simulation by modeling heterogeneous user behaviors and dynamic system response.

Trip Dynamics In the simulator, trips are not computed purely based on the Euclidean distance between origin and destination. The Euclidean distance d_{euclid} represents only the straight-line distance between two geographical points, but in real urban environments, vehicles must follow the road network, encounter intersections, traffic lights, and sometimes take detours. To capture this realism, the simulator models the **actual travel distance** as a stochastic extension of the Euclidean distance:

$$d_{\text{real}} = d_{\text{euclid}} \cdot (1 + \xi), \quad \xi \sim \text{Exp}(\text{DETOUR_SCALE}),$$

where ξ is an exponentially distributed random variable with scale parameter `DETOUR_SCALE`. This factor introduces random detours that account for non-optimal routing, road network complexity, and variability in user-selected paths.

The **travel time** is then modeled as:

$$t_{\text{travel}} = \frac{d_{\text{real}}}{v_{\text{avg}} \cdot (1 + \epsilon)}, \quad \epsilon \sim \mathcal{N}(0, \text{TRAFFIC_STD}),$$

where $v_{\text{avg}} = 40$ km/h is the average travel speed, and ϵ is a normally distributed noise term with standard deviation `TRAFFIC_STD`. This Gaussian term models random traffic conditions such as congestion, traffic lights, or varying driving behavior, resulting in fluctuations around the travel time.

Battery consumption is modeled as a linear function of the actual traveled distance:

$$\text{Battery Consumption} = 2 \cdot d_{\text{real}}$$

where each kilometer reduces the battery level by 2 units out of a total capacity of 150. This corresponds to an energy consumption of approximately:

$$\frac{2}{150} \times 100 = 1.33\% \text{ per kilometer,}$$

meaning that a fully charged vehicle can travel about 75 km before depletion. This formulation provides a realistic relationship between driving distance and energy usage, allowing the simulator to capture the impact of trip length and charging dynamics on fleet performance.

Departure Function. The **departure** process governs the dynamics following a successful trip assignment. Once a user is matched with a vehicle, a stochastic delay—sampled from an exponential distribution—is added to model the preparation and system-related waiting time before departure. The vehicle then starts the trip, whose duration depends on the travel distance, detour effects, and random traffic variations. At the end of the trip, the vehicle either becomes available again, enters a charging state if its battery is low, or is marked as unavailable when maintenance or charging capacity constraints occur.

Post-Trip Vehicle and Charging Behavior At the end of a trip, vehicles update their state:

- Vehicles with $B < 20\%$ start charging if a station is available; otherwise, they become unavailable until relocated.
- Vehicles with $B \geq 80\%$ return to the free state.
- Users may park at a charging station with probability $p = 0.4$ (`PROB_PARK_AT_STATION`).

Charging occurs at a linear rate until the battery reaches a threshold (`FREE_THRESHOLD_PERCENT`).

Fleet Relocation Vehicle relocation is scheduled at predefined hours (`RELOCATION_HOUR1 = 6`, `RELOCATION_HOUR2 = 15`) to ensure optimal fleet distribution and maintain service quality during peak demand periods. The relocation algorithm prioritizes vehicles with low battery levels or idle status, moving them to the nearest available charging station.

Vehicles are first filtered based on their state and battery level, then sorted according to a priority scheme:

- High-priority: battery level below 20%, requiring urgent relocation to prevent service interruption.

- Medium-priority: battery level between 20% and 50%, to preemptively support future demand.
- Low-priority: battery above 50%, only relocated if necessary to balance station occupancy.

For each candidate vehicle, the algorithm identifies the closest available charging station using Euclidean distance. Once a station is selected, the vehicle is assigned to it, starts the charging process, and its location is updated accordingly. This approach ensures that low-battery vehicles are systematically redistributed, minimizing service unavailability and supporting continuous fleet operation.

Event-Driven Simulation The simulator adopts a discrete-event approach based on a *Future Event Set* (FES), which is implemented as a **min-heap** priority queue using Python’s `heapq` library. The FES is stored as a list, where each element is an **Event** object containing:

- The scheduled event time (**time**)
- The event type (**type**), such as *user arrival*, *trip completion*, *charging end*, or *relocation*
- Optional data or object references (**data**) related to vehicles, users, or stations

Each time a new event is created, it is inserted into the FES through the `heapq.heappush()` operation, which maintains the heap property and ensures that the event with the smallest timestamp is always at the top of the queue. This mechanism allows events to be processed strictly in chronological order with logarithmic insertion and retrieval complexity.

The simulation loop continuously extracts the next event from the FES using `heapq.heappop()`, updates the simulation clock to the event time, executes the corresponding event handler, and schedules any resulting future events back into the queue.

Vehicles and stations are modeled as Python objects that store their current state, position, battery level, and destination. The use of dictionaries indexed by unique IDs allows efficient retrieval and modification of these objects during event processing. This architecture enables the simulator to reproduce realistic, stochastic system dynamics—such as demand variability, vehicle relocation, and charging behavior—without relying on fixed time-step updates.

3 Performance Analysis and Results Discussion

This section presents a comprehensive analysis of the simulator’s performance through quantitative metrics, graphical visualization, and statistical validation techniques. The objective is to evaluate service efficiency, fleet utilization, and system robustness under varying operational conditions.

The car-sharing simulation was run over a 72-hour period using a **Future Event Set (FES)** approach. At the end of each simulation run, the framework reports a set of aggregated indicators summarizing system performance.

In the standard scenario with **250 vehicles**, **400 stations**, a **pickup radius of 1.5 km**, and an **acceptance probability of 0.8** the service rate is about 65%. This value indicates a good overall performance, showing that the fleet size is adequate for the current demand. The **average availability rate** was 0.58 and the **utilization rate** 0.178, confirming a balanced distribution between free and active vehicles.

From the fleet-state evolution plot over 72 hours (three simulated days), free vehicles (green) start near 250 and gradually decrease as demand increases, while those in use (red) increase during peak hours 8–10 a.m. and 6–8 p.m.. Charging vehicles (blue) show distinct spikes at relocation hours, whereas unavailable ones (gray) rise slightly before relocation and then drop to zero. On the spatial map, three high-density clusters of stations correspond to main urban activity centers, confirming that hotspots were modeled in the areas of highest demand.

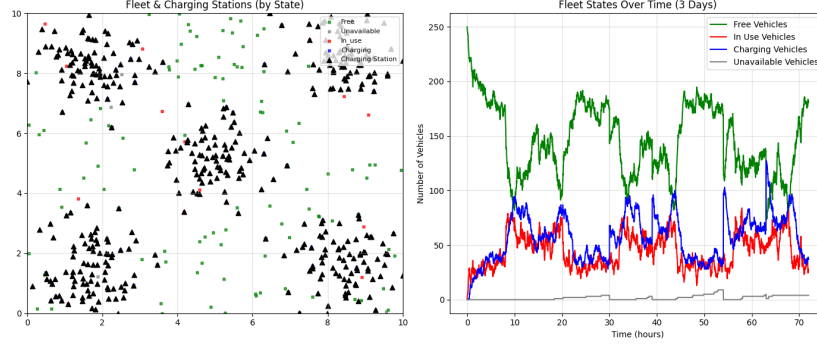


Figure 2: Fleet state evolution over time.

Trip metrics reveal that the simulator processed a total of 12,882 requests, of which 8,594 were served and 4,288 were dropped. Each trip covered an average distance of 5.70 km, with a mean travel time of 9.22 minutes. The average waiting time, including walking and system delays, was 13.22 minutes, yielding a total average usage time per request of 22.44 minutes. At the end of the simulation, the fleet consisted of 181 free vehicles, 27 in use, 37 charging, and 4 unavailable.

3.1 Temporal Demand Patterns

The hourly arrivals chart reveals peak demand periods:

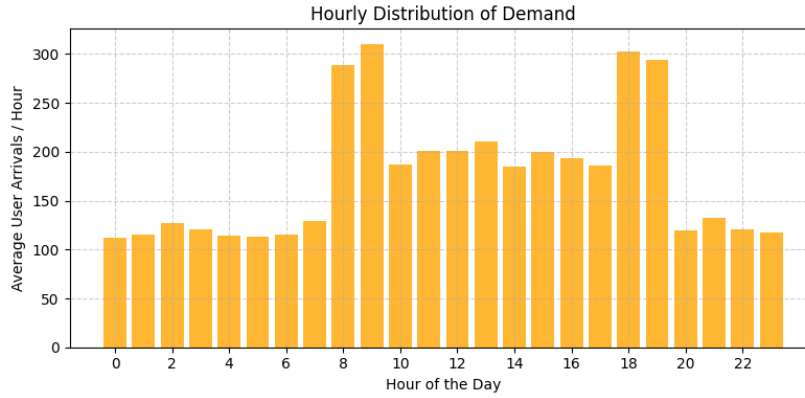


Figure 3: Hourly distribution of user demand.

The histogram of the hourly distribution of demand shows clear peaks during typical commuting and leisure periods. The first peak occurs between 8:00 and 10:00, corresponding to workers using the vehicles to go to their jobs. The second peak is observed between 18:00 and 20:00, when people return home from work or use the vehicles for leisure activities, hobbies, or errands. These peaks confirm the system experiences concentrated demand during standard rush hours and evening activity periods.

3.2 Insights and Reflections

1. **Battery threshold management:** Increasing the battery level required for a vehicle to be marked as **free** after charging improves service performance. Vehicles released at higher battery levels operate longer before becoming unavailable, resulting in higher service rates. Instead, if a vehicle is set as free at only 50% charge, it will deplete faster, leading to more dropped requests and lower system efficiency.
2. **Fleet sizing and pickup radius:** Larger fleets and greater pickup radius generally enhance service rates due to better coverage and improved vehicle availability near users.

3. **Charging strategy and station distribution:** The placement of stations and the relocation policy significantly affect utilization. Concentrating vehicles in hotspots improves response times, but overly centralized stations may leave peripheral areas underserved.
4. **System robustness:** Despite stochastic variations in traffic, detours, and user behavior, the system shows stable operational metrics and predictable service patterns under the tested parameters.
5. **Seed and Reproducibility:** To ensure reproducibility of the simulation results, a fixed random seed of 42 was used to initialize all stochastic components, including user arrivals, trip destinations, and relocation choices. Additional tests were conducted with different seed values, and the resulting performance metrics (e.g., service rate, waiting time, and utilization) showed only minor variations. This consistency confirms that the simulator produces stable and reproducible outcomes despite the inherent stochastic nature of the modeled system.

3.3 Grid Search and Hyperparameter Optimization

To evaluate the influence of configuration parameters on the system performance, a **grid search** was performed by systematically varying key parameters such as the battery weight, distance weight, minimum availability percentage, free threshold percentage, number of vehicles, number of stations, and base arrival rate per hour. Each simulation covered a 72-hour period and collected performance metrics including served and dropped requests, service rate, and utilization. The aim was to identify parameter combinations that maximize the service rate while preserving high fleet availability and balanced resource utilization.

Battery Weight	Distance Weight	Min. Avail. (%)	Free Threshold (%)	Vehicles	Stations	Service Rate
0.4	0.5	30	80	300	400	0.697
0.3	0.6	20	80	300	300	0.696
0.4	0.5	20	70	300	300	0.694
0.3	0.6	30	80	300	300	0.694
0.4	0.6	30	70	300	300	0.692
0.4	0.5	30	80	300	300	0.689
0.3	0.5	30	80	300	300	0.689
0.3	0.6	30	80	300	400	0.687
0.4	0.5	20	80	300	300	0.686
0.4	0.6	20	70	300	300	0.686

Table 1: Top 10 parameter combinations from the grid search, sorted by service rate.

The results in Table 1 show that the best-performing configurations generally feature a battery weight of 0.4 and a distance weight between 0.5 and 0.6, with minimum availability thresholds between 20% and 30% and free threshold levels around 70–80%. A fleet of 300 vehicles and a station network of 300–400 stations consistently yielded the highest service rates, approaching 70%. These findings suggest that a moderate emphasis on distance while maintaining sufficient battery availability enables more efficient fleet operations, balancing service rate, availability, and resource utilization across the system.

4 Lab 4: Transient phase detection & Confidence Intervals

4.1 Transient Detection

In the first part of the lab, the aim was to analyze the transient behavior of a single-server queue (M/M/1) with finite capacity ($N = 1000$) using a discrete-event simulation approach. The main goal was to detect the end of the transient phase and verify system performance under different arrival and service rate conditions.

Simulation Setup The simulation recorded the number of customers in the queue at each minute. A transient detection algorithm based on moving averages and a relative stability threshold was applied to automatically determine the end of the transient phase.

The detection algorithm operates as follows:

1. **Moving Window Calculation:** A moving average of the queue length is computed over a window of size w . To ensure adaptability across different simulation lengths, the window size is determined as a fixed percentage of the total dataset (e.g., 1%), with a minimum of 10 points to avoid instability in short simulations.
2. **Steady-State Identification:** The derivative of the moving average is compared against a tolerance threshold ε . The tolerance is initialized as 10% and automatically scaled by the mean of the values in the final window, ensuring consistency across different magnitudes of queue lengths. The transient is considered complete once the derivative remains below this threshold for the remaining simulation time.

This parameter configuration allows the detection process to remain robust and scalable, avoiding manual re-tuning across varying system sizes or simulation durations.

4.1.1 Results

Case 1: Stable Queue with Initial Full Capacity In this scenario, the queue starts at its maximum size (1000 customers) and quickly decreases as the service rate exceeds arrivals. The plot shows an exponential decay toward a steady-state level, confirming system stability. The transient phase is short, and the detect-transient method clearly identifies when the queue stabilizes, after which steady-state statistics and confidence intervals are reliable.

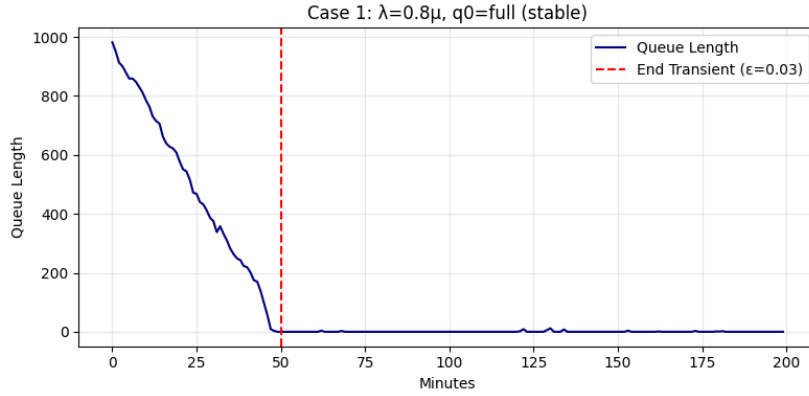


Figure 4: Arrival rate $\lambda = 0.8\mu$, with the queue initially full ($q_0 = 1000$). The vertical red dashed line indicates the detected end of the transient phase.

Case 2: Unstable Queue with Initial Empty State In this scenario, the arrival rate exceeds the service rate, so the queue grows rapidly from zero until it reaches the maximum capacity of 1000 customers. Once full, it remains constant, indicating that the system is saturated and unable to process incoming requests efficiently. The detect-transient method identifies a short transient phase followed by a steady but overloaded regime, reflecting a system operating well beyond its sustainable capacity.



Figure 5: Arrival rate $\lambda = 1.2\mu$, with the queue initially empty ($q_0 = 0$).

Case 3: Critical Queue ($\lambda = \mu$) In the critical case, the system operates at the boundary between stability and instability. The queue exhibits large fluctuations with slow convergence, often resembling a random walk. The transient period is long and poorly defined, making it difficult for the detection method to isolate a stationary region. As a result, the transient phase could not be reliably detected due to its high instability, making the estimates of steady-state performance difficult to determine.

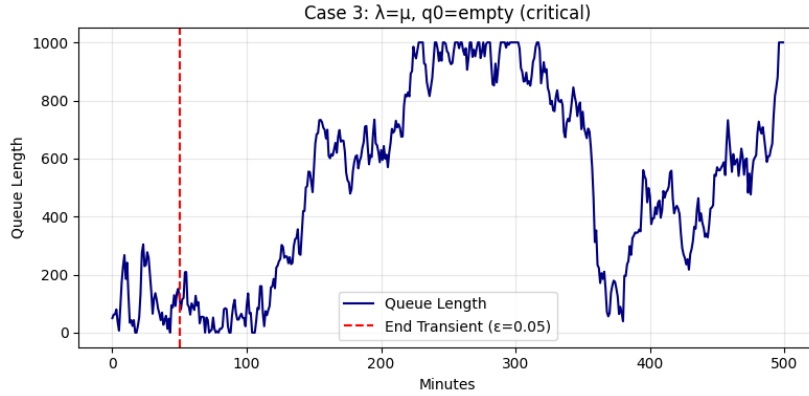


Figure 6: Queue length evolution for $\lambda = \mu$, with the queue initially empty ($q_0 = 0$). The vertical red dashed line marks the end of the transient phase.

4.1.2 Discussion

The transient detection algorithm proved effective in all three scenarios. It correctly identified the end of the transient phase even for initially overloaded or empty queues. Key observations include:

- In systems with a high initial queue length or when the system is stable ($\lambda < \mu$), the transient detection algorithm performs effectively, correctly identifying convergence toward steady state.
- When the queue capacity is limited, the transient detection also works well, as saturation dynamics lead to a stable operating regime even if $\lambda > \mu$.
- However, in the critical case ($\lambda = \mu$), the algorithm struggles to detect a clear transient period, since the system is marginally stable and no true steady state is reached.

These results demonstrate that the algorithm is robust and applicable to different operational conditions, providing reliable estimates for steady-state performance metrics.

4.2 Transient Phase Detection in Car-Sharing System

The transient detection algorithm used in the M/M/1 queue was also applied to our car-sharing system to analyze the dynamics of vehicles under different states. The aim is to detect the time after which the system reaches a quasi-stationary behavior, providing reliable metrics for fleet management and service planning.

Fleet Dynamics Overview The time series for the main fleet metrics show clear transient behaviors corresponding to critical hours of the day. For **Free Vehicles**, there are sharp drops around 8–10 a.m. and 6–8 p.m., reflecting periods of high demand, in addition to an initial transient at the start when all vehicles are available. **Vehicles in Use** exhibit peaks during the same morning and evening rush hours, consistent with the modeled demand patterns. **Charging Vehicles** show spikes in correspondence with relocation hours, when low-battery vehicles are systematically moved to charging stations.

These metrics are not quasi-stationary due to such temporal oscillations.

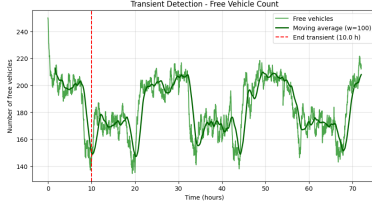


Figure 7: Free Vehicles

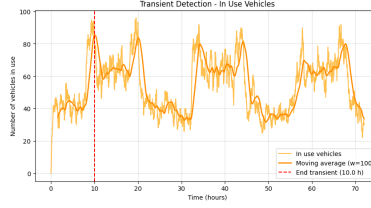


Figure 8: Vehicles in Use

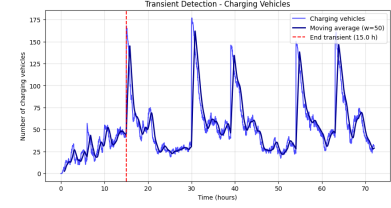


Figure 9: Charging Vehicles

Therefore, additional metrics with more stable behavior were considered to assess steady-state performance: **Mean Waiting Time (min)** and **Mean Duration (min)**. The **Mean Waiting Time (min)** and **Mean Duration (min)** are computed per served trip and are less affected by large temporal fluctuations. Mean Waiting Time indicates the typical delay experienced by users before starting a trip, reflecting system responsiveness, while Mean Total Usage captures the average duration of trips, indicating fleet utilization and user travel patterns.

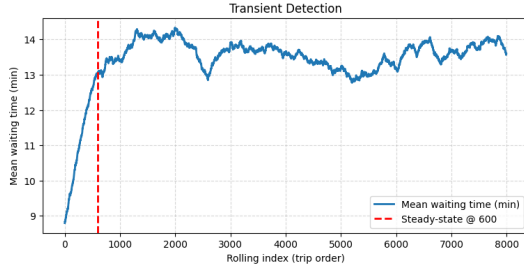


Figure 10: Mean Waiting Time.

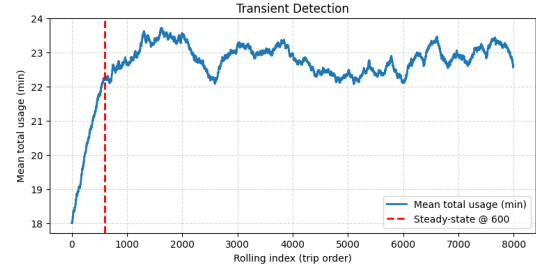


Figure 11: Mean Total Usage.

The trip-level metrics (**Mean Waiting Time** and **Mean Total Usage**) show a very similar behaviour, more stable, quasi-stationary after the initial transient, allowing reliable estimation of system performance.

Comparing the two approaches highlights that while fleet-level dynamics capture the temporal variability and operational effects of demand and relocations, per-trip metrics provide robust steady-state indicators for system efficiency and user experience.

4.3 Post-Transient Analysis and Confidence Intervals

To obtain meaningful performance metrics from a non-stationary system, a **transient detection** and **batch-based confidence interval computation** approach was applied to the fleet time series. After detecting the transient period, the goal is to compute reliable confidence intervals. To do so, the identified transient is removed from the time series before proceeding with the statistical analysis.

In the case of the fleet system, the time series for the main operational metrics exhibit strong daily oscillations rather than remaining quasi-stationary. Therefore, the analysis distinguishes between two scenarios:

Case 1 — Metrics with strong daily oscillations

Metrics such as **Free Vehicles**, **Vehicles in Use**, and **Charging Vehicles** show pronounced peaks and drops during daily rush hours and relocation periods, which violate the quasi-stationarity assumption. To isolate a stable portion of the process, the following parts are removed:

Initial transient + High-variability windows (8–10 a.m. and 6–8 p.m.).

Once the filtered time series is obtained, confidence intervals are computed as follows:

1. The time series is divided into **non-overlapping batches** of equal length.
2. The **mean** of each batch is computed.
3. The batch means are treated as approximately independent observations.
4. The overall mean and variance of the batch means are computed.
5. A **95% confidence interval** is obtained using the **Student’s t-distribution**.

Case 2 — Metrics that are already quasi-stationary

If a metric does not exhibit strong periodic oscillations, only the initial transient is removed. The remaining portion of the process is considered approximately stationary. The confidence interval computation proceeds as:

1. Remove the detected **initial transient**.
2. Partition the remaining time series into **equal-sized batches**.
3. Compute the **batch mean** for each batch.
4. Estimate the mean and variance from the set of batch means.
5. Construct the **95% confidence interval** using the **t-Student** formula.

4.3.1 Results

Table 2: Confidence Intervals (95%) — After Transient Removal

Metric	Mean	Lower	Upper	CI Width	Batches	Transient End
Free Vehicles	147.10	129.32	164.87	35.55	8	600
In Use Vehicles	43.51	36.29	50.73	14.44	8	600
Charging Vehicles	58.12	46.48	69.75	23.27	8	600
Mean Waiting Time (min)	13.37	13.03	13.71	0.69	41	200
Mean Total Usage (min)	22.61	22.24	22.98	0.74	41	200

The first three metrics show higher variability, with wider confidence intervals (35.55, 14.44, and 23.27), reflecting strong system dynamics. These pronounced oscillations indicate non-stationary behavior, leading to larger uncertainty in the mean estimates, which is consistent with what we expected.

In contrast, *Mean Waiting Time* and *Mean Total Usage* exhibit narrow confidence intervals (0.69 and 0.74) and quasi-stationary behavior after the initial transient. Their means are more stable and reliable. The observed difference aligns with our expectations, highlighting the importance of removing transients and focusing on stationary periods when evaluating system performance.

Conclusion

While the simulation captures key system dynamics, several improvements could enhance its realism. Extending the simulation period would help reveal how the system behaves in the longer run, incorporating additional operational constraints could refine vehicle and charging management, and using a more detailed city map would give more accurate insights into mobility patterns. Nonetheless, the model strikes a balance between complexity and interpretability: the results are quite strong and sufficient to capture the main fleet dynamics, yet simple enough to support general conclusions and guide strategic decision-making.

Overall, this approach shows how statistical analysis can inform operational strategies, identify potential bottlenecks and support decisions on fleet sizing, relocation policies, and charging infrastructure deployment.